🐔 Poultry Disease Detection — Model Training (Colab)

FYP: Poultry Disease Detection and Severity Assessment from Fecal Images

This notebook covers the whole ML side of your project, from raw images to a `.tflite` file for the Flutter app:

Step	What happens
1–3	Setup, config
4	Get the dataset, cache at 224×224, **find near-duplicates**, leak-free **train/val/test split**
5	Input pipeline + **augmentation (blur, brightness, noise)**
6	Train **MobileNetV2** (main) + **EfficientNetB0** (comparison), 2-stage transfer learning → evaluate (accuracy,
7	Choose which model to ship
8	Robustness test (dark / bright / blur / noise / low-res)
9	**Severity** rule (Mild / Moderate / Severe) + "not sure" rule
10	Export **TFLite** (fp32 / fp16 / int8-weights), verify it matches Keras, write files for Flutter
11	Optional: test on **your own local photos**

▶️ How to run (3 clicks)

1. **Runtime → Change runtime type → T4 GPU.**
2. *(Recommended first time)* In cell 3 set `QUICK_RUN = True` and **Runtime → Run all**. This is a rehearsal on a small subset (a few minutes of training) that proves everything works in your Colab. The first run still has to download the dataset once (~8.5 GB, usually 5–15 min); it is cached on Drive for later runs.
3. Set `QUICK_RUN = False` and **Run all** again for the real training. Nothing to upload: the dataset is **downloaded automatically** from Zenodo (no login, checksums verified). Expect roughly 45–90 minutes the first time (download + one-time resize are the slow parts) and ~30 minutes on later sessions.

Disconnects are safe: the resized dataset is cached on your Drive (`PoultryFYP/cache224.tar`), so after a disconnect the next run skips the download and resize. Results are saved to `PoultryFYP/outputs/`. If you don't mount Drive, everything still runs but is saved in temporary storage — the last cell lets you download a zip.

(If you would rather use the Kaggle copy of the dataset: upload its zip to Drive as `PoultryFYP/dataset.zip`; `DATASET_SOURCE = "auto"` will then use it instead of Zenodo.)

1) Check GPU and versions

```
#@title 1) Check GPU and versions
import tensorflow as tf, keras
print("TensorFlow", tf.__version__, "| Keras", keras.__version__)
gpus = tf.config.list_physical_devices("GPU")
print("GPU:", gpus if gpus else "NONE  ->  Runtime > Change runtime type > T
```

```
TensorFlow 2.20.0 | Keras 3.13.2
GPU: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

2) Mount Google Drive (results are saved there so a disconnect never loses them)

```
#@title 2) Mount Google Drive (results are saved there so a disconnect never
try:
    from google.colab import drive
    drive.mount("/content/drive")
except Exception as e:
    print("Drive NOT mounted -> results will be saved in /content/PoultryFYP
```

```
Drive NOT mounted -> results will be saved in /content/PoultryFYP (temporary!
```

3) Configuration — the only cell you may need to edit

```
#@title 3) Configuration — the only cell you may need to edit
import os, re, json, random, time, shutil, zipfile, subprocess, hashlib, pic
from pathlib import Path
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf, keras
from keras import layers

# ------------------------------------------------------------------ run mod
# True  = 5-10 minute rehearsal (small subset, few epochs) -> checks that ev
# False = the real training run.
QUICK_RUN = False

# ------------------------------------------------------------------ paths
_HAS_DRIVE = os.path.isdir("/content/drive/MyDrive")
PROJECT_DIR    = "/content/drive/MyDrive/PoultryFYP" if _HAS_DRIVE else "/co
OUT_DIR        = f"{PROJECT_DIR}/outputs" + ("_quick" if QUICK_RUN else "")
LOCAL_TEST_DIR = f"{PROJECT_DIR}/local_test"            # optional: your own
CACHE_ARCHIVE  = f"{PROJECT_DIR}/cache224.tar"          # resized dataset ke
INDEX_PKL      = f"{PROJECT_DIR}/cache224_index.pkl"
RAW_DIR        = "/content/raw"                         # temporary (fast lo
CACHE_DIR      = "/content/cache224"                    # temporary
RUN_DIR        = "/content/runs"                        # temporary

# ------------------------------------------------------------------ dataset
# "auto"     : use PoultryFYP/dataset.zip on Drive if it exists, otherwise d
```

```
# "zenodo"   : always download the 4 class zips from Zenodo (checksums verif
# "drive_zip": use DATASET_ZIP (e.g. the Kaggle zip you uploaded)
# "kaggle"   : Kaggle CLI (needs kaggle.json)
# "folder"   : already unzipped somewhere -> set DATASET_FOLDER
DATASET_SOURCE = "auto"
DATASET_ZIP    = f"{PROJECT_DIR}/dataset.zip"
DATASET_FOLDER = "/content/raw"
ZENODO_BASE    = "https://zenodo.org/records/4628934/files"      # Machuve e
ZENODO_MD5     = {"cocci": "0f0c308c48463c70b51cc3719e3a35c0", "healthy": "5
                  "ncd": "293b3781f163b4a1cc6a25c3244690c0",   "salmo": "333

# ------------------------------------------------------------------ classes
# ORDER MATTERS: this is the model output order and the order in labels.txt
CLASS_NAMES = ["Coccidiosis", "Healthy", "Newcastle", "Salmonella"]
NUM_CLASSES = len(CLASS_NAMES)
CLASS_TO_ID = {c: i for i, c in enumerate(CLASS_NAMES)}
ALIASES = {  # folder-name -> class   (matching ignores case, spaces, punctu
    "cocci": "Coccidiosis", "coccidiosis": "Coccidiosis",
    "healthy": "Healthy", "normal": "Healthy",
    "ncd": "Newcastle", "newcastle": "Newcastle", "newcastledisease": "Newca
    "salmo": "Salmonella", "salmonella": "Salmonella",
}

# ------------------------------------------------------------------ trainin
RUN_MODELS      = ["mobilenetv2", "efficientnetb0"]  # trained + compared. A
EXPORT_MODEL    = "mobilenetv2"     # the one converted to TFLite for the ap
IMG_SIZE        = 224
BATCH_SIZE      = 32
SEED            = 42
USE_PRETRAINED  = True              # ImageNet weights (keep True; False onl
HEAD_EPOCHS     = 8                 # stage 1: base frozen, train the new he
HEAD_LR         = 1e-3
FT_EPOCHS       = 25                # stage 2: fine-tune top of the base (ea
FT_LR           = 5e-5
UNFREEZE_FRAC   = 0.40              # fraction of the base layers (from the
LABEL_SMOOTHING = 0.05
CLASS_WEIGHT_POWER = 0.5            # 0 = no weighting, 1 = fully "balanced"
DUP_HASH_THRESHOLD = 16             # near-duplicate = <=16 differing bits o
QUICK_PER_CLASS = 250               # images per class used when QUICK_RUN=T

if QUICK_RUN:
    RUN_MODELS, EXPORT_MODEL = ["mobilenetv2"], "mobilenetv2"
    HEAD_EPOCHS, FT_EPOCHS = 2, 3

keras.utils.set_random_seed(SEED)
os.makedirs(OUT_DIR, exist_ok=True); os.makedirs(RUN_DIR, exist_ok=True)
print(("QUICK RUN (rehearsal)" if QUICK_RUN else "FULL RUN"), "| Drive:", "y
```

```
FULL RUN | Drive: NO (temporary storage) | outputs -> /content/PoultryFYP/out
```

4) Dataset → index → cache → leak-free split

Public benchmarks for this dataset report 97–99 % accuracy, but many use a *random* split. In this dataset several photos of the same sample/session look almost identical, so a random split leaks near-copies from train into test and inflates accuracy (Chidziwisano et al. 2025 show accuracy dropping from 91 % to 69 % on data from another country). So here we **(a)** find near-duplicate images, **(b)** keep each duplicate group entirely inside one split, and **(c)** report that honest number.

4a) Get the images (auto-download) and index them

```
#@title 4a) Get the images (auto-download) and index them
from PIL import Image, ImageOps
try:
    from tqdm.auto import tqdm
except Exception:
    tqdm = lambda x, **k: x
Image.MAX_IMAGE_PIXELS = None
IMG_EXT = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}

def _md5(path, blk=1 << 22):
    h = hashlib.md5()
    with open(path, "rb") as f:
        for b in iter(lambda: f.read(blk), b""): h.update(b)
    return h.hexdigest()

def download_zenodo():
    from concurrent.futures import ThreadPoolExecutor
    def get(name):
        z = f"{RAW_DIR}/{name}.zip"
        for attempt in range(1, 4):
            subprocess.run(["wget", "-c", "-q", "--tries=3", "--timeout=60",
            if os.path.exists(z) and _md5(z) == ZENODO_MD5[name]: return nam
            print(f"  {name}: checksum/download problem (attempt {attempt}/3
            if os.path.exists(z) and attempt < 3: os.remove(z)
        return name, False
    print("Downloading 4 zips from Zenodo in parallel (~8.5 GB, usually 5-15
    with ThreadPoolExecutor(max_workers=4) as ex:
        futs = [ex.submit(get, n) for n in ZENODO_MD5]
        while not all(f.done() for f in futs):
            gb = sum(os.path.getsize(f"{RAW_DIR}/{n}.zip") for n in ZENODO_M
            print(f"  downloaded so far: {gb:.2f} GB", end="\r"); time.sleep
        results = [f.result() for f in futs]
    for name, ok in results:
        if not ok: print(f"WARNING: {name}.zip did not pass the checksum - w
    def extract(name):
        z = f"{RAW_DIR}/{name}.zip"; dst = f"{RAW_DIR}/{name}"; os.makedirs(
        with zipfile.ZipFile(z) as zf:
            zf.extractall(dst, members=[m for m in zf.namelist() if "__MACOS
        os.remove(z); return name
    print("\nExtracting...")
    with ThreadPoolExecutor(max_workers=4) as ex: [print("  extracted", n) f
```

```
def prepare_raw_data():
    src = DATASET_SOURCE
    if src == "folder": return DATASET_FOLDER
    if src == "auto": src = "drive_zip" if os.path.exists(DATASET_ZIP) else
    os.makedirs(RAW_DIR, exist_ok=True)
    marker = Path(RAW_DIR) / ".ready"
    if marker.exists(): print("Raw images already prepared in", RAW_DIR); re
    print("Dataset source:", src)
    if src == "drive_zip":
        print("Unzipping", DATASET_ZIP, "...")
        with zipfile.ZipFile(DATASET_ZIP) as z: z.extractall(RAW_DIR)
    elif src == "kaggle":
        subprocess.run(["kaggle", "datasets", "download", "-d", "allandclive
    elif src == "zenodo":
        download_zenodo()
    else:
        raise ValueError("Unknown DATASET_SOURCE: " + src)
    marker.write_text("ok"); return RAW_DIR

def _key(s): return re.sub(r"[^a-z0-9]", "", s.lower())

def index_images(root):
    # Label = nearest parent folder whose name is a known class alias.
    root_p = Path(root); rows = []
    files = [p for p in root_p.rglob("*") if p.is_file() and p.suffix.lower(
             and not p.name.startswith(".") and "__MACOSX" not in p.parts]
    for p in files:
        for parent in p.parents:
            if parent == root_p: break
            if _key(parent.name) in ALIASES:
                rows.append((str(p), ALIASES[_key(parent.name)])); break
    if len(rows) < 50:   # fallback: a CSV with image + label columns (e.g.
        for csv in root_p.rglob("*.csv"):
            try: c = pd.read_csv(csv)
            except Exception: continue
            cols = {x.lower(): x for x in c.columns}
            lab = next((cols[k] for k in cols if k in ("label", "class", "la
            img = next((cols[k] for k in cols if k in ("image", "images", "f
            if lab and img:
                lut = {os.path.basename(str(a)): _key(str(b)) for a, b in zi
                rows = [(str(p), ALIASES[lut[p.name]]) for p in files if lut
                if rows: print("Labels taken from", csv.name); break
    df = pd.DataFrame(rows, columns=["src", "label"])
    if df.empty:
        raise RuntimeError(f"No labelled images found under {root}. Folder n
    return df

# ---- if a previous session already prepared the data, restore it from Driv
RESTORED = False
if (not QUICK_RUN) and os.path.exists(INDEX_PKL) and os.path.exists(CACHE_AR
    print("Found a cached dataset on Drive -> restoring (no download / resiz
    with tarfile.open(CACHE_ARCHIVE) as tf_: tf_.extractall(os.path.dirname(
    _saved = pickle.load(open(INDEX_PKL, "rb")); df, H = _saved["df"], _save
    print(f"Restored {len(df)} images.")
else:
```

```
    root = prepare_raw_data()
    df_all = index_images(root)
    if QUICK_RUN:
        df_all = df_all.sample(frac=1, random_state=SEED).groupby("label").h
    print(f"Found {len(df_all)} labelled images")
    print(df_all["label"].value_counts().reindex(CLASS_NAMES).to_string())
```

```
Dataset source: zenodo
Downloading 4 zips from Zenodo in parallel (~8.5 GB, usually 5-15 min)...

Extracting...
  extracted cocci
  extracted healthy
  extracted ncd
  extracted salmo
Found 6812 labelled images
label
Coccidiosis    2103
Healthy        2057
Newcastle       376
Salmonella     2276
```

4b) Resize to 224x224 once (fast training) + compute duplicate fingerprints

```
#@title 4b) Resize to 224x224 once (fast training) + compute duplicate finge
def load_resized(path, size=IMG_SIZE):
    # SAME function is used for training data and for your local-photo test,
    im = Image.open(path)
    try: im.draft("RGB", (size * 2, size * 2))      # fast JPEG down-scaling
    except Exception: pass
    im = ImageOps.exif_transpose(im).convert("RGB")
    return im.resize((size, size), Image.BILINEAR)   # direct resize, no cro

def fingerprints(im, hs=16):
    # 256-bit difference-hash, for the image and its h-flip, v-flip, both fl
    g = np.asarray(im.convert("L").resize((hs + 1, hs), Image.BILINEAR), dty
    out = []
    for gg in (g, g[:, ::-1], g[::-1, :], g[::-1, ::-1]):
        out.append(np.packbits((gg[:, 1:] > gg[:, :-1]).flatten()))   # 32 b
    return np.stack(out)                                               # (4,

def _work(args):
    i, src, label = args
    try:
        im = load_resized(src)
        dst = f"{CACHE_DIR}/{label}/{i:05d}.jpg"
        os.makedirs(os.path.dirname(dst), exist_ok=True)
        im.save(dst, quality=95)
        return i, dst, fingerprints(im)
    except Exception as e:
        return i, None, str(e)

if RESTORED:
```

```
        print("Skipped - using restored cache.")
    else:
        from concurrent.futures import ThreadPoolExecutor
        t0 = time.time()
        jobs = [(i, r.src, r.label) for i, r in enumerate(df_all.itertuples(inde
        with ThreadPoolExecutor(max_workers=8) as ex:
            results = list(tqdm(ex.map(_work, jobs), total=len(jobs)))
        bad = [r for r in results if r[1] is None]
        good = [r for r in results if r[1] is not None]
        print(f"Cached {len(good)} images in {time.time()-t0:.0f}s | unreadable/
        df = pd.DataFrame({"path": [g[1] for g in good],
                           "src":  [df_all.src.iloc[g[0]] for g in good],
                           "label":[df_all.label.iloc[g[0]] for g in good]})
        df["y"] = df["label"].map(CLASS_TO_ID)
        H = np.stack([g[2] for g in good])          # (n, 4, 32) uint8
        if not QUICK_RUN:                            # keep the resized dataset
            try:
                with tarfile.open(CACHE_ARCHIVE, "w") as t_: t_.add(CACHE_DIR, a
                pickle.dump(dict(df=df, H=H), open(INDEX_PKL, "wb"))
                print(f"Saved cache to Drive ({os.path.getsize(CACHE_ARCHIVE)/1e
            except Exception as e:
                print("Could not save cache to Drive (not fatal):", e)
    print(df["label"].value_counts().reindex(CLASS_NAMES).to_string())
```

```
100%                                                 6812/6812 [03:02<00:00, 21.63it/s]
Cached 6812 images in 183s | unreadable/corrupt skipped: 0
Saved cache to Drive (167 MB) - next session skips download + resize.
label
Coccidiosis    2103
Healthy        2057
Newcastle       376
Salmonella     2276
```

4c) Find near-duplicate images and build a leak-free split (train ~70% / val ~15% / test ~15%)

```
#@title 4c) Find near-duplicate images and build a leak-free split (train ~7
POP8 = np.array([bin(i).count("1") for i in range(256)], dtype=np.uint8)

def duplicate_groups(H, thr, chunk=128):
    n = H.shape[0]; parent = np.arange(n)
    def find(a):
        while parent[a] != a:
            parent[a] = parent[parent[a]]; a = parent[a]
        return a
    ref = H[:, 0, :]
    for s in range(0, n, chunk):
        blk = ref[s:s + chunk]
        dmin = np.full((len(blk), n), 255, dtype=np.int32)
        for v in range(H.shape[1]):
            d = POP8[np.bitwise_xor(blk[:, None, :], H[None, :, v, :])].sum(
            dmin = np.minimum(dmin, d)
        ii, jj = np.nonzero(dmin <= thr)
```

```
            for a, b in zip(ii + s, jj):
                if a < b:
                    ra, rb = find(a), find(b)
                    if ra != rb: parent[rb] = ra
        return np.array([find(i) for i in range(n)])

    t0 = time.time()
    df["group"] = duplicate_groups(H, DUP_HASH_THRESHOLD)
    sizes = df.groupby("group")["y"].transform("size")
    n_multi = int((sizes > 1).sum())
    print(f"Duplicate search took {time.time()-t0:.0f}s: {df['group'].nunique()}
          f"{n_multi} images ({100*n_multi/len(df):.1f}%) belong to a near-dupli

    # a group that contains images with DIFFERENT labels is label noise -> drop
    conflict = df.groupby("group")["y"].transform("nunique") > 1
    if conflict.any():
        print(f"Dropping {int(conflict.sum())} images whose near-duplicates carr
        df = df[~conflict].reset_index(drop=True)   # (fingerprints H are not ne

    from sklearn.model_selection import StratifiedGroupKFold
    y_all, g_all = df["y"].values, df["group"].values
    sgk = StratifiedGroupKFold(n_splits=7, shuffle=True, random_state=SEED)
    rest_idx, test_idx = next(iter(sgk.split(np.zeros(len(df)), y_all, g_all)))
    sgk2 = StratifiedGroupKFold(n_splits=6, shuffle=True, random_state=SEED)
    tr_rel, va_rel = next(iter(sgk2.split(np.zeros(len(rest_idx)), y_all[rest_id
    train_idx, val_idx = rest_idx[tr_rel], rest_idx[va_rel]

    df["split"] = ""
    df.loc[train_idx, "split"] = "train"; df.loc[val_idx, "split"] = "val"; df.l
    assert not (set(df.loc[train_idx, "group"]) & set(df.loc[test_idx, "group"])
    assert not (set(df.loc[train_idx, "group"]) & set(df.loc[val_idx, "group"]))
    assert not (set(df.loc[val_idx, "group"]) & set(df.loc[test_idx, "group"])),

    tab = pd.crosstab(df["label"], df["split"]).reindex(CLASS_NAMES)[["train", "
    tab.loc["TOTAL"] = tab.sum()
    print("\nImages per split (no duplicate group crosses a split):"); print(tab
    df.drop(columns=["src"]).to_csv(f"{OUT_DIR}/split.csv", index=False)   # kee
    tr_df, va_df, te_df = (df[df.split == s].reset_index(drop=True) for s in ("t
```

```
Duplicate search took 27s: 6494 unique groups; 578 images (8.5%) belong to a

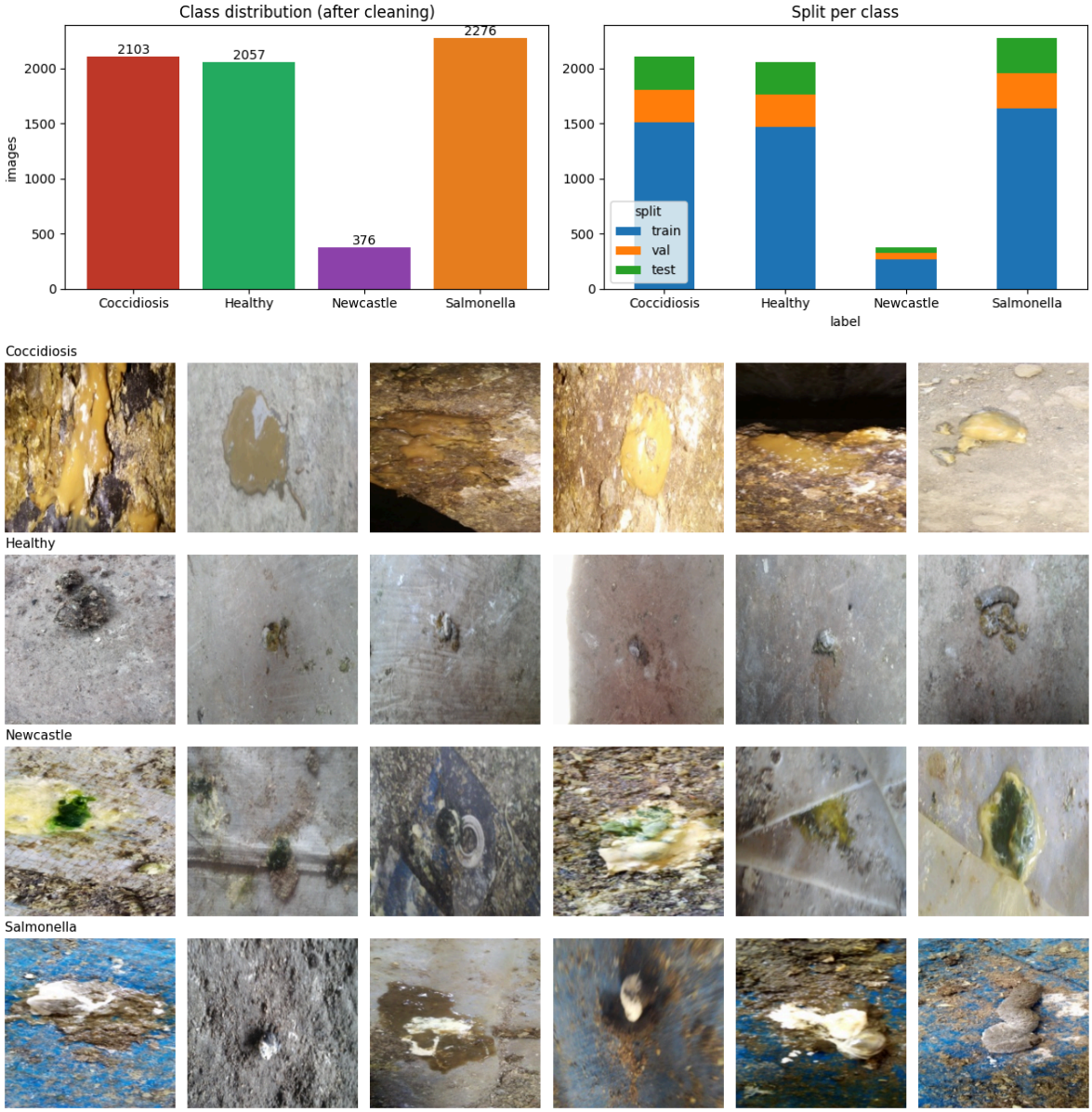
Images per split (no duplicate group crosses a split):
split        train  val  test
label
Coccidiosis   1509  298   296
Healthy       1471  293   293
Newcastle      266   56    54
Salmonella    1638  315   323
TOTAL         4884  962   966
```

4d) Look at the data (figures for your report)

```
#@title 4d) Look at the data (figures for your report)
counts = df["label"].value_counts().reindex(CLASS_NAMES)
```

```
fig, ax = plt.subplots(1, 2, figsize=(12, 3.8))
ax[0].bar(counts.index, counts.values, color=["#c0392b", "#27ae60", "#8e44ad
for i, v in enumerate(counts.values): ax[0].text(i, v, str(v), ha="center",
ax[0].set_title("Class distribution (after cleaning)"); ax[0].set_ylabel("im
tab.drop("TOTAL").plot(kind="bar", stacked=True, ax=ax[1], rot=0); ax[1].set
plt.tight_layout(); plt.savefig(f"{OUT_DIR}/fig_class_distribution.png", dpi

fig, axes = plt.subplots(NUM_CLASSES, 6, figsize=(13, 2.3 * NUM_CLASSES))
rng = np.random.default_rng(SEED)
for r, c in enumerate(CLASS_NAMES):
    sub = tr_df[tr_df.label == c]
    for k, idx in enumerate(rng.choice(len(sub), size=min(6, len(sub)), repl
        axes[r, k].imshow(Image.open(sub.path.iloc[idx])); axes[r, k].axis("
    axes[r, 0].set_title(c, loc="left", fontsize=11)
plt.tight_layout(); plt.savefig(f"{OUT_DIR}/fig_samples.png", dpi=120); plt.
```



5) Input pipeline and augmentation

Your documentation (FR-10) asks for **blur, brightness variation and controlled noise**. Those are implemented below, together with flips / small rotations / zoom (a dropping has no "up" direction).

Colour is deliberately *not* jittered strongly (no hue shift): colour is one of the main visual cues of these diseases, so destroying it would hurt the model. Only mild saturation changes are used.

Augmentation runs on the **training set only**; validation and test images are never modified.

5) tf.data pipeline + augmentation

```
#@title 5) tf.data pipeline + augmentation
AUTOTUNE = tf.data.AUTOTUNE

# class weights from TRAIN counts only (dampened by CLASS_WEIGHT_POWER)
cnt = tr_df["y"].value_counts().reindex(range(NUM_CLASSES)).values.astype("f
w = (cnt.sum() / (NUM_CLASSES * cnt)) ** CLASS_WEIGHT_POWER
w = w / (w * cnt).sum() * cnt.sum()                        # mean sample weig
CLASS_W = tf.constant(w, tf.float32)
print("class weights:", {c: round(float(x), 2) for c, x in zip(CLASS_NAMES,

_flip  = layers.RandomFlip("horizontal_and_vertical")
_rot   = layers.RandomRotation(0.055, fill_mode="reflect")            # abou
_zoom  = layers.RandomZoom((-0.15, 0.15), fill_mode="reflect")
_shift = layers.RandomTranslation(0.08, 0.08, fill_mode="reflect")

def gaussian_blur(x, sigma, k=7):
    ax = tf.range(-(k // 2), k // 2 + 1, dtype=tf.float32)
    g = tf.exp(-(ax ** 2) / (2.0 * sigma ** 2)); g = g / tf.reduce_sum(g)
    kern = tf.tile((g[:, None] * g[None, :])[:, :, None, None], [1, 1, 3, 1]
    return tf.nn.depthwise_conv2d(x, kern, [1, 1, 1, 1], "SAME")

def augment_batch(x, y):
    x = tf.cast(x, tf.float32)
    x = _flip(x, training=True); x = _rot(x, training=True)
    x = _zoom(x, training=True); x = _shift(x, training=True)
    B = tf.shape(x)[0]
    u = lambda lo, hi: tf.random.uniform([B, 1, 1, 1], lo, hi)
    coin = lambda p: tf.cast(tf.random.uniform([B, 1, 1, 1]) < p, tf.float32
    x = x * u(0.75, 1.25)                                              # bri
    m = tf.reduce_mean(x, axis=[1, 2, 3], keepdims=True)
    x = (x - m) * u(0.8, 1.2) + m                                      # con
    gray = tf.reduce_mean(x, axis=-1, keepdims=True)
    x = gray + (x - gray) * u(0.85, 1.15)                              # mil
    x = x + tf.random.normal(tf.shape(x)) * u(0.0, 10.0) * coin(0.5)   # gau
    sigma = tf.random.uniform([], 0.6, 1.8)
    mb = coin(0.3)
    x = mb * gaussian_blur(x, sigma) + (1.0 - mb) * x                  # blu
    return tf.clip_by_value(x, 0.0, 255.0), y

def _decode(path):
```

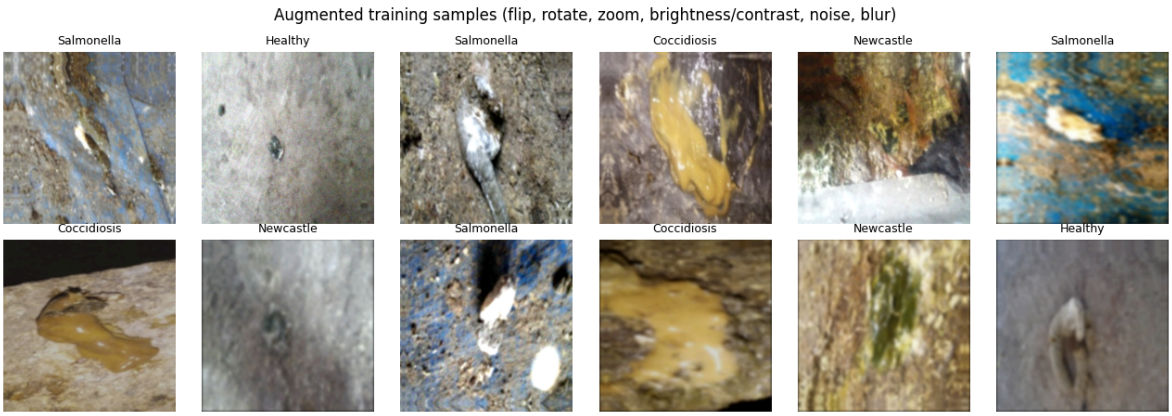
```
    img = tf.io.decode_jpeg(tf.io.read_file(path), channels=3)
    img.set_shape([IMG_SIZE, IMG_SIZE, 3]); return img                # uin

def make_ds(frame, training):
    ds = tf.data.Dataset.from_tensor_slices((frame["path"].values, frame["y"
    ds = ds.map(lambda p, y: (_decode(p), y), num_parallel_calls=AUTOTUNE).c
    if training:
        ds = ds.shuffle(len(frame), seed=SEED, reshuffle_each_iteration=True
        ds = ds.map(augment_batch, num_parallel_calls=AUTOTUNE)
        ds = ds.map(lambda x, y: (x, tf.one_hot(y, NUM_CLASSES), tf.gather(C
    else:
        ds = ds.batch(BATCH_SIZE)
        ds = ds.map(lambda x, y: (tf.cast(x, tf.float32), tf.one_hot(y, NUM_
    return ds.prefetch(AUTOTUNE)

train_ds, val_ds, test_ds = make_ds(tr_df, True), make_ds(va_df, False), mak

# save an augmentation preview for the report
xb, yb, _ = next(iter(train_ds.take(1)))
fig, axes = plt.subplots(2, 6, figsize=(13, 4.6))
for i, a in enumerate(axes.flat):
    a.imshow(xb[i].numpy().astype("uint8")); a.set_title(CLASS_NAMES[int(np.
plt.suptitle("Augmented training samples (flip, rotate, zoom, brightness/con
plt.tight_layout(); plt.savefig(f"{OUT_DIR}/fig_augmentation.png", dpi=130);
```

```
class weights: {'Coccidiosis': 0.94, 'Healthy': 0.95, 'Newcastle': 2.24, 'Sal
```



6) Models and training

Why MobileNetV2 is the main model: in the papers you collected it is the model most often recommended for *deployment* on this exact dataset — Machuve et al. 2022 (98.02 % after fine-tuning, chosen for its light weight and generalisation), Tasdelen & Arslan 2025 (best of six, 97.1 %, also the fastest to train), Li et al. 2023 (modified MobileNetV2, <1 s per image on-site) and Pranta 2025 (61 ms/image). It is about 14 MB (3.5 MB quantised), which suits mid-range Android phones (NFR-01). **EfficientNetB0** is trained as the accuracy challenger (Dhungana et al. 2025: 99.12 %);

Xception / ResNet50 are the heavy models in the literature and are too big for a phone, so they are only useful as an optional baseline.

Paper (from your research folder)	
Machuve et al. 2022	After fine-tuning (BatchNorm frozen): MobileNetV2 98.02 %, Xception 98.24
Tasdelen & Arslan 2025	MobileNetV2 best of six models (97.1 %) and fastest to train
Pranta 2025	MobileNetV2 + SVM 96.17 %, 61 ms/image; MobileNetV3-Small only up to 83.'
Dhungana et al. 2025	EfficientNet-B0 99.12 % (with YOLO11n crop + augmentation)
Degu & Simegn 2023	ResNet50 98.7 %
Chidziwisano et al. 2025	0.91 on Tanzania test data → 0.69 on unseen Malawi data

*Training recipe (from Machuve et al.): stage 1 trains only the new classifier head; stage 2 fine-tunes the top of the network with a small learning rate while **BatchNorm layers stay frozen**.*

The image pre-processing (scaling to what the network expects) is **inside the model**, so the app just feeds raw RGB values 0–255.

6a) Model builder + two-stage training function

```
#@title 6a) Model builder + two-stage training function
def _xception(**kw): return keras.applications.Xception(**kw)
MODEL_ZOO = {   # name -> (constructor, scaling done by us)   ("none" = the
    "mobilenetv2":    (keras.applications.MobileNetV2,    "minus1_1"),
    "efficientnetb0": (keras.applications.EfficientNetB0, "none"),
    "xception":       (_xception,                          "minus1_1"),
}

def build_model(name):
    ctor, pre = MODEL_ZOO[name]
    inputs = keras.Input((IMG_SIZE, IMG_SIZE, 3), name="image")          # f
    x = layers.Rescaling(1 / 127.5, offset=-1, name="to_minus1_1")(inputs) i
    base = ctor(include_top=False, weights="imagenet" if USE_PRETRAINED else
                input_shape=(IMG_SIZE, IMG_SIZE, 3))
    base.trainable = False
    x = base(x, training=False)                                          # B
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.25)(x)
    x = layers.Dense(128, activation="relu")(x)
    x = layers.Dropout(0.25)(x)
    logits = layers.Dense(NUM_CLASSES, name="logits")(x)
    probs = layers.Softmax(name="probs")(logits)
    return keras.Model(inputs, probs, name=name), base

def set_finetune(base, frac):
    base.trainable = True
    n = len(base.layers); cut = int(n * (1 - frac))
    for i, l in enumerate(base.layers):
        l.trainable = (i >= cut) and not isinstance(l, layers.BatchNormaliza
    print(f"fine-tuning {sum(l.trainable for l in base.layers)}/{n} base lay
```

```
def compile_model(model, lr):
    model.compile(optimizer=keras.optimizers.Adam(lr),
                  loss=keras.losses.CategoricalCrossentropy(label_smoothing=
                  metrics=["accuracy"])

def train_model(name):
    print(f"\n{'='*70}\nTraining {name}\n{'='*70}")
    keras.utils.set_random_seed(SEED)
    model, base = build_model(name)
    run = f"{RUN_DIR}/{name}"; os.makedirs(run, exist_ok=True)
    compile_model(model, HEAD_LR)
    h1 = model.fit(train_ds, validation_data=val_ds, epochs=HEAD_EPOCHS, shu
    set_finetune(base, UNFREEZE_FRAC)
    compile_model(model, FT_LR)
    cbs = [keras.callbacks.ModelCheckpoint(f"{run}/best.keras", monitor="val
           keras.callbacks.EarlyStopping(monitor="val_loss", patience=6, res
           keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.3,
    h2 = model.fit(train_ds, validation_data=val_ds, initial_epoch=HEAD_EPOC
                   epochs=HEAD_EPOCHS + FT_EPOCHS, callbacks=cbs, shuffle=Fa
    hist = {k: h1.history[k] + h2.history[k] for k in h1.history}
    pd.DataFrame(hist).to_csv(f"{OUT_DIR}/history_{name}.csv", index=False)
    fig, ax = plt.subplots(1, 2, figsize=(11, 3.5))
    for a, k, t in zip(ax, ["loss", "accuracy"], ["Loss", "Accuracy"]):
        a.plot(hist[k], label="train"); a.plot(hist["val_" + k], label="val"
        a.axvline(HEAD_EPOCHS - 0.5, ls="--", c="gray"); a.set_title(f"{name
    plt.tight_layout(); plt.savefig(f"{OUT_DIR}/fig_curves_{name}.png", dpi=
    return model
```

6b) Evaluation + calibration helpers

```
#@title 6b) Evaluation + calibration helpers
from sklearn.metrics import (classification_report, confusion_matrix, accura
                             f1_score, roc_auc_score)

def softmax_np(z):
    z = z - z.max(1, keepdims=True); e = np.exp(z); return e / e.sum(1, keep

def get_logits(model, ds, lm=None):
    lm = lm or keras.Model(model.input, model.get_layer("logits").output)
    return lm.predict(ds, verbose=0)

def fit_temperature(logits, y):
    from scipy.optimize import minimize_scalar
    def nll(T):
        z = logits / T; z = z - z.max(1, keepdims=True)
        logp = z - np.log(np.exp(z).sum(1, keepdims=True))
        return -logp[np.arange(len(y)), y].mean()
    return float(minimize_scalar(nll, bounds=(0.05, 10.0), method="bounded")

def ece_score(probs, y, bins=15):
    conf, pred = probs.max(1), probs.argmax(1); acc = pred == y; e = 0.0
    edges = np.linspace(0, 1, bins + 1)
    for lo, hi in zip(edges[:-1], edges[1:]):
```

```
            m = (conf > lo) & (conf <= hi)
            if m.any(): e += m.mean() * abs(acc[m].mean() - conf[m].mean())
        return float(e)

    def plot_eval(probs, y, name, tag):
        pred = probs.argmax(1)
        cm = confusion_matrix(y, pred, labels=range(NUM_CLASSES))
        cmn = cm / np.maximum(cm.sum(1, keepdims=True), 1)
        fig, ax = plt.subplots(1, 3, figsize=(17, 4.6))
        for a, M, ttl, fmt in [(ax[0], cm, "Confusion matrix (counts)", "d"), (a
            im = a.imshow(M, cmap="Blues"); a.set_title(f"{name}: {ttl}")
            a.set_xticks(range(NUM_CLASSES)); a.set_xticklabels(CLASS_NAMES, rot
            a.set_xlabel("predicted"); a.set_ylabel("true")
            for i in range(NUM_CLASSES):
                for j in range(NUM_CLASSES):
                    a.text(j, i, format(M[i, j], fmt), ha="center", va="center",
        conf = probs.max(1); acc = (pred == y); edges = np.linspace(0, 1, 11); x
        for lo, hi in zip(edges[:-1], edges[1:]):
            m = (conf > lo) & (conf <= hi)
            if m.sum() >= 5: xs.append(conf[m].mean()); ys.append(acc[m].mean())
        ax[2].plot([0, 1], [0, 1], "--", c="gray"); ax[2].plot(xs, ys, "o-"); ax
        ax[2].set_xlabel("mean confidence"); ax[2].set_ylabel("actual accuracy")
        plt.tight_layout(); plt.savefig(f"{OUT_DIR}/fig_eval_{name}.png", dpi=13

    def evaluate_and_calibrate(model, name):
        # 1) fit temperature on VALIDATION logits  2) bake it into the last laye
        y_va, y_te = va_df["y"].values, te_df["y"].values
        lm = keras.Model(model.input, model.get_layer("logits").output)
        T = fit_temperature(get_logits(model, val_ds, lm), y_va)
        lg_te = get_logits(model, test_ds, lm)
        p_raw, p_cal = softmax_np(lg_te), softmax_np(lg_te / T)
        layer = model.get_layer("logits"); W, b = layer.get_weights(); layer.set
        chk = model.predict(test_ds, verbose=0)
        assert np.allclose(chk, p_cal, atol=1e-4), "temperature baking mismatch"
        pred = p_cal.argmax(1)
        rep = classification_report(y_te, pred, target_names=CLASS_NAMES, output
        print(classification_report(y_te, pred, target_names=CLASS_NAMES, digits
        try: auc = float(roc_auc_score(y_te, p_cal, multi_class="ovr"))
        except Exception: auc = float("nan")
        res = dict(model=name, params_M=round(model.count_params() / 1e6, 2), te
                   macro_f1=f1_score(y_te, pred, average="macro"), roc_auc_ovr=a
                   newcastle_recall=rep["Newcastle"]["recall"], temperature=T,
                   ece_before=ece_score(p_raw, y_te), ece_after=ece_score(p_cal,
        print({k: (round(v, 4) if isinstance(v, float) else v) for k, v in res.i
        plot_eval(p_cal, y_te, name, "calibrated")
        json.dump(dict(summary=res, per_class=rep), open(f"{OUT_DIR}/metrics_{na
        model.save(f"{OUT_DIR}/{name}_calibrated.keras")
        return res, p_cal
```

6c) TRAIN (this is the long cell) — trains every model in RUN_MODELS, then evaluates on the test set

```
#@title 6c) TRAIN (this is the long cell) — trains every model in RUN_MODELS
TRAINED, RESULTS, TEST_PROBS = {}, [], {}
for name in RUN_MODELS:
    m = train_model(name)
    res, p = evaluate_and_calibrate(m, name)
    TRAINED[name], TEST_PROBS[name] = m, p
    RESULTS.append(res)
cmp_df = pd.DataFrame(RESULTS)
cmp_df.to_csv(f"{OUT_DIR}/model_comparison.csv", index=False)
cmp_df.round(4)
```

```
======================================================================
Training mobilenetv2
======================================================================
Downloading data from https://storage.googleapis.com/tensorflow/keras-applica
9406464/9406464 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1/8
153/153 - 272s - 2s/step - accuracy: 0.8200 - loss: 0.6788 - val_accuracy: 0.
Epoch 2/8
153/153 - 210s - 1s/step - accuracy: 0.8860 - loss: 0.5253 - val_accuracy: 0.
Epoch 3/8
153/153 - 212s - 1s/step - accuracy: 0.8982 - loss: 0.4844 - val_accuracy: 0.
Epoch 4/8
153/153 - 261s - 2s/step - accuracy: 0.9038 - loss: 0.4649 - val_accuracy: 0.
Epoch 5/8
153/153 - 210s - 1s/step - accuracy: 0.9048 - loss: 0.4662 - val_accuracy: 0.
Epoch 6/8
153/153 - 213s - 1s/step - accuracy: 0.9130 - loss: 0.4481 - val_accuracy: 0.
Epoch 7/8
153/153 - 207s - 1s/step - accuracy: 0.9107 - loss: 0.4463 - val_accuracy: 0.
Epoch 8/8
153/153 - 261s - 2s/step - accuracy: 0.9124 - loss: 0.4418 - val_accuracy: 0.
fine-tuning 41/154 base layers (BatchNorm frozen)
Epoch 9/33
153/153 - 242s - 2s/step - accuracy: 0.9066 - loss: 0.4799 - val_accuracy: 0.
Epoch 10/33
153/153 - 209s - 1s/step - accuracy: 0.9216 - loss: 0.4251 - val_accuracy: 0.
Epoch 11/33
153/153 - 210s - 1s/step - accuracy: 0.9433 - loss: 0.3800 - val_accuracy: 0.
Epoch 12/33

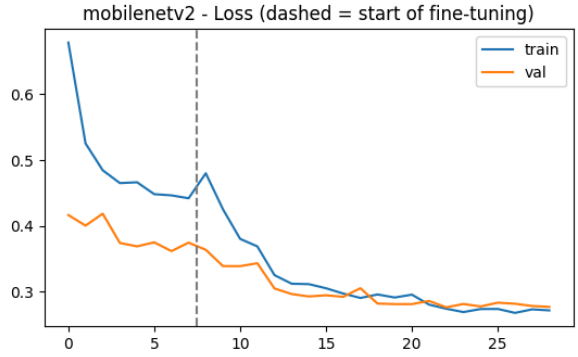
Epoch 12: ReduceLROnPlateau reducing learning rate to 1.4999999621068127e-05.
153/153 - 207s - 1s/step - accuracy: 0.9433 - loss: 0.3686 - val_accuracy: 0.
Epoch 13/33
153/153 - 208s - 1s/step - accuracy: 0.9642 - loss: 0.3249 - val_accuracy: 0.
Epoch 14/33
153/153 - 210s - 1s/step - accuracy: 0.9681 - loss: 0.3118 - val_accuracy: 0.
Epoch 15/33
153/153 - 211s - 1s/step - accuracy: 0.9623 - loss: 0.3111 - val_accuracy: 0.
Epoch 16/33
153/153 - 208s - 1s/step - accuracy: 0.9662 - loss: 0.3049 - val_accuracy: 0.
Epoch 17/33
153/153 - 262s - 2s/step - accuracy: 0.9730 - loss: 0.2968 - val_accuracy: 0.
Epoch 18/33
153/153 - 210s - 1s/step - accuracy: 0.9734 - loss: 0.2901 - val_accuracy: 0.
Epoch 19/33
153/153 - 208s - 1s/step - accuracy: 0.9695 - loss: 0.2954 - val_accuracy: 0.
Epoch 20/33
153/153 - 261s - 2s/step - accuracy: 0.9746 - loss: 0.2908 - val_accuracy: 0.
Epoch 21/33
153/153 - 205s - 1s/step - accuracy: 0.9709 - loss: 0.2951 - val_accuracy: 0.
Epoch 22/33

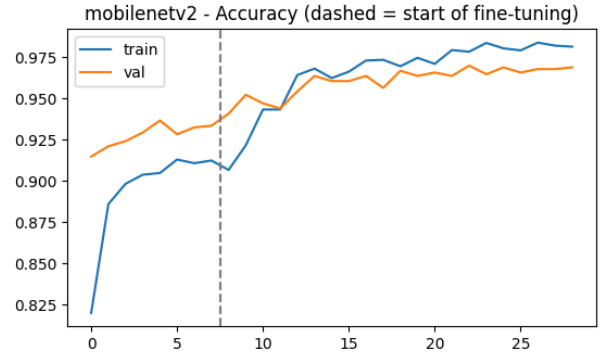
Epoch 22: ReduceLROnPlateau reducing learning rate to 4.499999886320438e-06.
153/153 - 208s - 1s/step - accuracy: 0.9793 - loss: 0.2800 - val_accuracy: 0.
Epoch 23/33
153/153 - 208s - 1s/step - accuracy: 0.9783 - loss: 0.2737 - val_accuracy: 0.
Epoch 24/33
153/153 - 207s - 1s/step - accuracy: 0.9836 - loss: 0.2686 - val_accuracy: 0.
Epoch 25/33
```

```
Epoch 25: ReduceLROnPlateau reducing learning rate to 1.3499999113264492e-06.
153/153 - 266s - 2s/step - accuracy: 0.9803 - loss: 0.2732 - val_accuracy: 0.
Epoch 26/33
153/153 - 261s - 2s/step - accuracy: 0.9791 - loss: 0.2734 - val_accuracy: 0.
Epoch 27/33

Epoch 27: ReduceLROnPlateau reducing learning rate to 4.0499998021914507e-07.
153/153 - 213s - 1s/step - accuracy: 0.9838 - loss: 0.2674 - val_accuracy: 0.
Epoch 28/33
153/153 - 211s - 1s/step - accuracy: 0.9820 - loss: 0.2727 - val_accuracy: 0.
Epoch 29/33

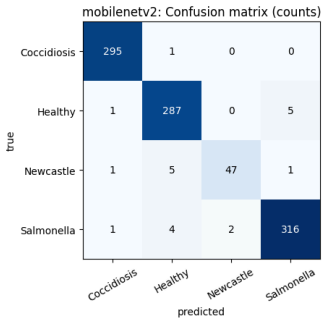
Epoch 29: ReduceLROnPlateau reducing learning rate to 1.2149999406574352e-07.
153/153 - 210s - 1s/step - accuracy: 0.9814 - loss: 0.2712 - val_accuracy: 0.
```

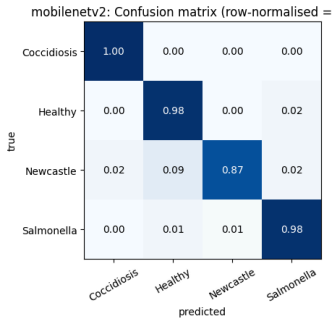


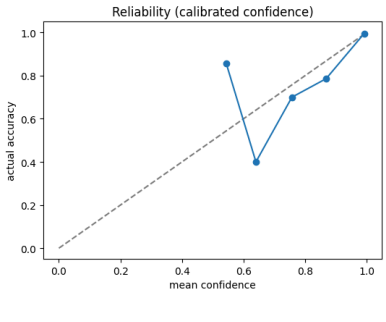


	precision	recall	f1-score	support
Coccidiosis	0.9899	0.9966	0.9933	296
Healthy	0.9663	0.9795	0.9729	293
Newcastle	0.9592	0.8704	0.9126	54
Salmonella	0.9814	0.9783	0.9798	323
accuracy			0.9783	966
macro avg	0.9742	0.9562	0.9647	966
weighted avg	0.9782	0.9783	0.9781	966

```
{'model': 'mobilenetv2', 'params_M': 2.42, 'test_acc': 0.9783, 'macro_f1': 0.
```








```
==========================================================================
Training efficientnetb0
==========================================================================
Downloading data from https://storage.googleapis.com/keras-applications/effic
16705208/16705208 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1/8
153/153 - 270s - 2s/step - accuracy: 0.8593 - loss: 0.5633 - val_accuracy: 0.
Epoch 2/8
153/153 - 208s - 1s/step - accuracy: 0.9163 - loss: 0.4306 - val_accuracy: 0.
Epoch 3/8
153/153 - 258s - 2s/step - accuracy: 0.9292 - loss: 0.4015 - val_accuracy: 0.
Epoch 4/8
```

```
153/153 - 204s - 1s/step - accuracy: 0.9363 - loss: 0.3883 - val_accuracy: 0.
Epoch 5/8
153/153 - 265s - 2s/step - accuracy: 0.9406 - loss: 0.3819 - val_accuracy: 0.
Epoch 6/8
153/153 - 205s - 1s/step - accuracy: 0.9400 - loss: 0.3726 - val_accuracy: 0.
Epoch 7/8
153/153 - 263s - 2s/step - accuracy: 0.9412 - loss: 0.3686 - val_accuracy: 0.
Epoch 8/8
153/153 - 261s - 2s/step - accuracy: 0.9445 - loss: 0.3668 - val_accuracy: 0.
fine-tuning 76/238 base layers (BatchNorm frozen)
Epoch 9/33
153/153 - 263s - 2s/step - accuracy: 0.9607 - loss: 0.3308 - val_accuracy: 0.
Epoch 10/33
153/153 - 202s - 1s/step - accuracy: 0.9722 - loss: 0.3016 - val_accuracy: 0.
Epoch 11/33
153/153 - 204s - 1s/step - accuracy: 0.9785 - loss: 0.2918 - val_accuracy: 0.
Epoch 12/33
153/153 - 261s - 2s/step - accuracy: 0.9797 - loss: 0.2792 - val_accuracy: 0.
Epoch 13/33
153/153 - 205s - 1s/step - accuracy: 0.9838 - loss: 0.2739 - val_accuracy: 0.
Epoch 14/33
153/153 - 205s - 1s/step - accuracy: 0.9846 - loss: 0.2696 - val_accuracy: 0.
Epoch 15/33
153/153 - 203s - 1s/step - accuracy: 0.9865 - loss: 0.2597 - val_accuracy: 0.
Epoch 16/33
153/153 - 205s - 1s/step - accuracy: 0.9869 - loss: 0.2582 - val_accuracy: 0.
Epoch 17/33
153/153 - 203s - 1s/step - accuracy: 0.9896 - loss: 0.2530 - val_accuracy: 0.
Epoch 18/33
153/153 - 203s - 1s/step - accuracy: 0.9869 - loss: 0.2507 - val_accuracy: 0.
Epoch 19/33

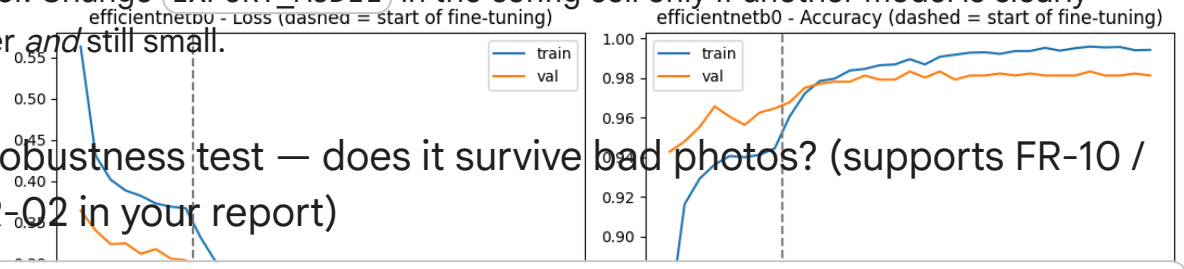
Epoch 19: ReduceLROnPlateau reducing learning rate to 1.4999999621068127e-05.
153/153 - 202s - 1s/step - accuracy: 0.9908 - loss: 0.2466 - val_accuracy: 0.
Epoch 20/33
153/153 - 203s - 1s/step - accuracy: 0.9918 - loss: 0.2424 - val_accuracy: 0.
Epoch 21/33

Epoch 21: ReduceLROnPlateau reducing learning rate to 4.499999886320438e-06.
153/153 - 202s - 1s/step - accuracy: 0.9928 - loss: 0.2410 - val_accuracy: 0.
Epoch 22/33
153/153 - 201s - 1s/step - accuracy: 0.9930 - loss: 0.2389 - val_accuracy: 0.
Epoch 23/33
153/153 - 202s - 1s/step - accuracy: 0.9922 - loss: 0.2420 - val_accuracy: 0.
Epoch 24/33
153/153 - 201s - 1s/step - accuracy: 0.9937 - loss: 0.2390 - val_accuracy: 0.
Epoch 25/33
153/153 - 202s - 1s/step - accuracy: 0.9937 - loss: 0.2373 - val_accuracy: 0.
Epoch 26/33
153/153 - 203s - 1s/step - accuracy: 0.9953 - loss: 0.2356 - val_accuracy: 0.
Epoch 27/33

Epoch 27: ReduceLROnPlateau reducing learning rate to 1.3499999113264492e-06.
153/153 - 201s - 1s/step - accuracy: 0.9939 - loss: 0.2380 - val_accuracy: 0.
Epoch 28/33
153/153 - 203s - 1s/step - accuracy: 0.9951 - loss: 0.2363 - val_accuracy: 0.
Epoch 29/33
153/153 - 203s - 1s/step - accuracy: 0.9959 - loss: 0.2347 - val_accuracy: 0.
Epoch 30/33
153/153 - 201s - 1s/step - accuracy: 0.9955 - loss: 0.2359 - val_accuracy: 0.
Epoch 31/33
```

7) Pick the model to ship

```
Epoch 31: ReduceLROnPlateau reducing learning rate to 4.0499998021914507e-07.
153/153 - 201s - 1s/step - accuracy: 0.9957 - loss: 0.2359 - val_accuracy: 0.
Epoch 32/33
153/153 - 203s - 1s/step - accuracy: 0.9941 - loss: 0.2366 - val_accuracy: 0.
Epoch 33/33
Epoch 33: ReduceLROnPlateau reducing learning rate to 1.2149999405574352e-07.
153/153 - 2  - accuracy: 0.9943 - loss: 0.2352 - val_accuracy: 0.
```



Look at the comparison table above. Rule of thumb for an Android FYP: if the two models are within ~1 point of each other, **ship the lighter one (MobileNetV2)** — it is smaller and faster on a mid-range phone, which is exactly what NFR-01 and NFR-08 ask for. Change `EXPORT_MODEL` in the config cell only if another model is clearly better *and* still small.

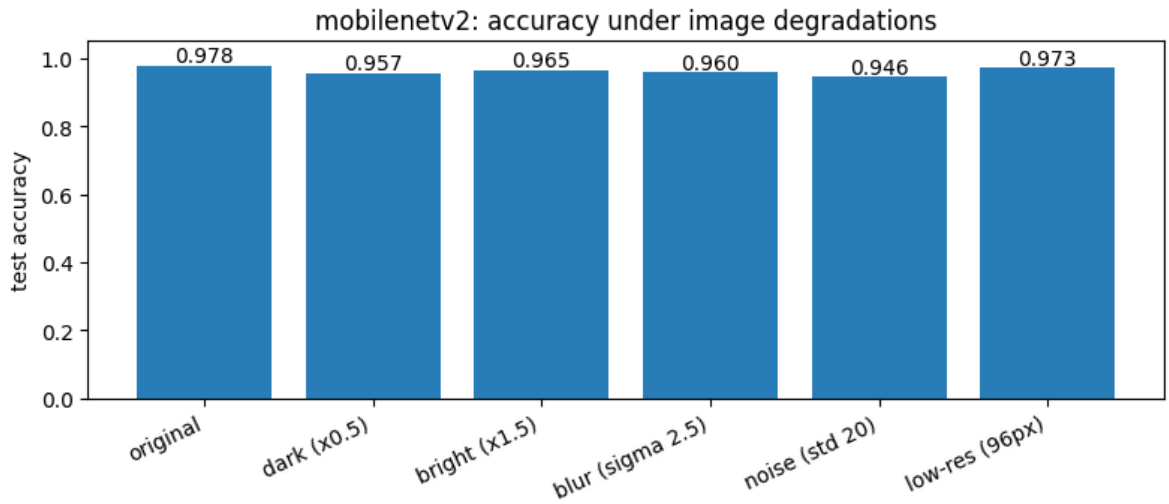
8) Robustness test — does it survive bad photos? (supports FR-10 / NFR-02 in your report)

```
#@title 8) Robustness test — does it survive bad photos? (supports FR-10 / N
def load_u8(paths): return np.stack([np.asarray(Image.open(p).convert("RGB")
X_te = load_u8(te_df["path"]); y_te = te_df["y"].values
model = TRAINED[EXPORT_MODEL]

PERTURB = {
    "original":        lambda x: x,
    "dark (x0.5)":     lambda x: x * 0.5,
    "bright (x1.5)":   lambda x: tf.clip_by_value(x * 1.5, 0, 255),
    "blur (sigma 2.5)": lambda x: gaussian_blur(x, 2.5),
    "noise (std 20)":  lambda x: tf.clip_by_value(x + tf.random.normal(tf.sh
    "low-res (96px)":  lambda x: tf.image.resize(tf.image.resize(x, [96, 96]
}
rows = []
for k, fn in PERTURB.items():
    tf.random.set_seed(SEED); preds = []
    for i in range(0, len(X_te), 64):
        xb = fn(tf.cast(X_te[i:i + 64], tf.float32))
        preds.append(model.predict(xb, verbose=0).argmax(1))
    preds = np.concatenate(preds)
    rows.append(dict(condition=k, accuracy=accuracy_score(y_te, preds), macr
rob = pd.DataFrame(rows); rob.to_csv(f"{OUT_DIR}/robustness_{EXPORT_MODEL}.c
plt.figure(figsize=(8, 3.6)); plt.bar(rob.condition, rob.accuracy, color="#2
plt.ylim(0, 1.05); plt.ylabel("test accuracy"); plt.title(f"{EXPORT_MODEL}:
for i, v in enumerate(rob.accuracy): plt.text(i, v + 0.01, f"{v:.3f}", ha="c
plt.tight_layout(); plt.savefig(f"{OUT_DIR}/fig_robustness.png", dpi=130); p
rob.round(4)
```

Show 25 per page

Like what you see? Visit the data table notebook to learn more about interactive tables.



	condition	accuracy	macro_f1
0	original	0.9783	0.9647
1	dark (x0.5)	0.9565	0.9334
2	bright (x1.5)	0.9648	0.9448
3	blur (sigma 2.5)	0.9596	0.9387
4	noise (std 20)	0.9462	0.9132
5	low-res (96px)	0.9731	0.9513

9) Severity (Mild / Moderate / Severe) and the "not sure" rule

Your design says severity is **derived from the confidence score and the predicted class** (FR-05). The code below does exactly that, but on the **calibrated** confidence from step 6, so 0.90 really means "right about 90 % of the time".

- Confidence below `uncertain_below` → the app should say **"Not sure — retake the photo / see a vet"** instead of guessing. This protects against blurry photos and images that are not droppings at all.
- Above that, the range up to 100 % is split into three equal bands: Mild → Moderate → Severe.
- `min_severity_by_class`: Newcastle disease is highly contagious, so it is never shown as Mild.
- **Healthy** has no severity.

The threshold is chosen on the **validation** set (never on test) as the lowest confidence at which the model is ≥ 95 % accurate.

> ⚠️ **Be honest about this in your report/viva:** a classifier's confidence is not a clinical measurement of severity. Call it a *model-certainty-based risk level* and list it as a limitation (real severity grading needs veterinary-labelled severity data).

9) Choose thresholds on VALIDATION data, write severity_config.json, test on TEST data

```
#@title 9) Choose thresholds on VALIDATION data, write severity_config.json,
SEV = ["Mild", "Moderate", "Severe"]
p_val = model.predict(val_ds, verbose=0); y_va = va_df["y"].values
conf_v, ok_v = p_val.max(1), (p_val.argmax(1) == y_va)

sugg = None
for t in np.arange(0.50, 0.901, 0.05):
    m = conf_v >= t
    if m.sum() >= 30 and ok_v[m].mean() >= 0.95: sugg = round(float(t), 2);
u = float(np.clip(sugg if sugg is not None else 0.60, 0.50, 0.80))
span = (1 - u) / 3
SEV_CFG = {"uncertain_below": round(u, 2), "mild_max": round(u + span, 2), "
           "min_severity_by_class": {"Newcastle": "Moderate"},
           "note": "Severity is a model-certainty-based risk level, not a cl
print("Chosen from validation data:", SEV_CFG)

def assess(probs, cfg=SEV_CFG):
    k = int(np.argmax(probs)); conf = float(probs[k]); label = CLASS_NAMES[k
    if conf < cfg["uncertain_below"]:
        return dict(status="uncertain", predicted_class=label, confidence=co
    if label == "Healthy":
        return dict(status="healthy", predicted_class=label, confidence=conf
    sev = "Mild" if conf < cfg["mild_max"] else ("Moderate" if conf < cfg["m
    floor = cfg["min_severity_by_class"].get(label)
    if floor and SEV.index(sev) < SEV.index(floor): sev = floor
    return dict(status="disease", predicted_class=label, confidence=conf, se

p_te = model.predict(test_ds, verbose=0)
rows = [assess(p) for p in p_te]
res = pd.DataFrame(rows); res["correct"] = (p_te.argmax(1) == y_te)
acc_ = res[res.status != "uncertain"]
print(f"\nTEST: {100*(res.status!='uncertain').mean():.1f}% of photos get a
      f"accuracy on those = {100*acc_.correct.mean():.2f}%  (accuracy on ALL
print("Uncertain rate per true class:")
print(pd.Series((res.status == "uncertain").values, index=[CLASS_NAMES[i] fo
print("\nSeverity bands on TEST (disease predictions):")
band = res[res.status == "disease"].groupby("severity")["correct"].agg(n="si
print(band.round(3).to_string())
json.dump(SEV_CFG, open(f"{OUT_DIR}/severity_config.json", "w"), indent=2)
print("\nExamples:"); [print(" ", {k: (round(v, 3) if isinstance(v, float) e
```

```
Chosen from validation data: {'uncertain_below': 0.5, 'mild_max': 0.67, 'mode

TEST: 99.7% of photos get a confident answer; accuracy on those = 98.13%  (ac
```

```
Uncertain rate per true class:
Coccidiosis    0.003
Healthy        0.003
Newcastle      0.019
Salmonella     0.000

Severity bands on TEST (disease predictions):
            n  accuracy
severity
Mild        8     0.750
Moderate   10     0.700
Severe    649     0.994

Examples:
  {'status': 'disease', 'predicted_class': 'Salmonella', 'confidence': 0.999,
  {'status': 'disease', 'predicted_class': 'Salmonella', 'confidence': 0.999,
  {'status': 'disease', 'predicted_class': 'Salmonella', 'confidence': 0.976,
  {'status': 'disease', 'predicted_class': 'Salmonella', 'confidence': 0.991,
  {'status': 'disease', 'predicted_class': 'Salmonella', 'confidence': 0.978,
[None, None, None, None, None]
```

10) Export to TensorFlow Lite and verify it

Three variants are produced. Every one is **re-evaluated on the test set** to prove the phone version behaves like the Keras version (a common source of "it worked in Colab but not in the app" bugs). The **fp16** variant is chosen by default (half the size of fp32, no speed penalty on phones) as long as it loses ≤ 0.5 accuracy points; otherwise fp32. The `dynamic` (int8-weights) file is even smaller but can be *slower* on some CPUs, so it is left as a manual option.

10) Convert -> TFLite (fp32 / fp16 / dynamic-range) and check accuracy + size + speed

```
#@title 10) Convert -> TFLite (fp32 / fp16 / dynamic-range) and check accura
DEPLOY = f"{OUT_DIR}/deploy"; VAR_DIR = f"{DEPLOY}/variants"
os.makedirs(VAR_DIR, exist_ok=True)

def convert_all(model, name):
    sm = f"{RUN_DIR}/{name}_saved_model"
    if os.path.exists(sm): shutil.rmtree(sm)
    model.export(sm)                                    # Keras 3 -> TF SavedM
    out = {}
    def conv(tag, fn):
        c = tf.lite.TFLiteConverter.from_saved_model(sm); fn(c)
        path = f"{VAR_DIR}/{name}_{tag}.tflite"; open(path, "wb").write(c.co
    def fp16(c): c.optimizations = [tf.lite.Optimize.DEFAULT]; c.target_spec
    def dyn(c):  c.optimizations = [tf.lite.Optimize.DEFAULT]
    conv("fp32", lambda c: None); conv("fp16", fp16); conv("dynamic", dyn)
    return out
```

```
try:                                    # newer, recommended runtime (tf.lite
    from ai_edge_litert.interpreter import Interpreter as TFLiteInterpreter
except Exception:
    TFLiteInterpreter = tf.lite.Interpreter

def tflite_predict(path, X_u8, threads=2):
    it = TFLiteInterpreter(model_path=path, num_threads=threads); it.allocat
    i_in, i_out = it.get_input_details()[0], it.get_output_details()[0]
    probs = np.zeros((len(X_u8), NUM_CLASSES), np.float32); t0 = time.perf_c
    for i, x in enumerate(X_u8):
        it.set_tensor(i_in["index"], x[None].astype(np.float32)); it.invoke(
        probs[i] = it.get_tensor(i_out["index"])[0]
    return probs, (time.perf_counter() - t0) / len(X_u8) * 1000.0, i_in

variants = convert_all(model, EXPORT_MODEL)
keras_acc = accuracy_score(y_te, p_te.argmax(1)); rows = []
for tag, path in variants.items():
    pr, ms, i_in = tflite_predict(path, X_te)
    rows.append(dict(variant=tag, size_MB=round(os.path.getsize(path) / 1e6,
                     macro_f1=f1_score(y_te, pr.argmax(1), average="macro"),
                     agree_with_keras=float((pr.argmax(1) == p_te.argmax(1))
                     max_prob_diff=float(np.abs(pr - p_te).max()), ms_per_im
tfl = pd.DataFrame(rows); tfl.to_csv(f"{OUT_DIR}/tflite_comparison.csv", ind
print(f"Keras test accuracy: {keras_acc:.4f}   | TFLite input: shape {[int(v
print(tfl.round(4).to_string(index=False))
print("(ms/image here is the Colab CPU - a real phone will differ; measure i

# fp16 = half the size, same speed on phones, practically identical results
# ("dynamic" is smaller still but can be SLOWER on some CPUs, so it is only
fp16_ok = float(tfl.set_index("variant").loc["fp16", "test_acc"]) >= keras_a
CHOSEN = "fp16" if fp16_ok else "fp32"
print("\n=> Chosen variant for the app:", CHOSEN, "(copy another one from th
final_path = f"{DEPLOY}/poultry_{EXPORT_MODEL}_{CHOSEN}.tflite"
shutil.copy(variants[CHOSEN], final_path)

# ---- files the Flutter app needs
open(f"{DEPLOY}/labels.txt", "w").write("\n".join(CLASS_NAMES) + "\n")
json.dump(SEV_CFG, open(f"{DEPLOY}/severity_config.json", "w"), indent=2)
T_used = float(pd.DataFrame(RESULTS).set_index("model").loc[EXPORT_MODEL, "t
meta = dict(model=EXPORT_MODEL, file=os.path.basename(final_path), variant=C
            input=dict(shape=[1, IMG_SIZE, IMG_SIZE, 3], dtype="float32", co
                       resize="direct resize to 224x224, no crop / no paddin
            output="softmax probabilities, same order as labels", calibratio
            test_accuracy=float(tfl.set_index("variant").loc[CHOSEN, "test_a
            test_macro_f1=float(tfl.set_index("variant").loc[CHOSEN, "macro_
json.dump(meta, open(f"{DEPLOY}/model_meta.json", "w"), indent=2)

# ---- parity samples: 2 test images per class + the probabilities the app M
os.makedirs(f"{DEPLOY}/parity", exist_ok=True); par = []
pr_final, _, _ = tflite_predict(final_path, X_te)
for c in range(NUM_CLASSES):
    for j in np.where(y_te == c)[0][:2]:
        fn = f"{CLASS_NAMES[c]}_{j}.jpg"; shutil.copy(te_df.path.iloc[j], f"
        par.append(dict(file=fn, true=CLASS_NAMES[c], probs=[round(float(v),
```

```
json.dump(par, open(f"{DEPLOY}/parity/expected.json", "w"), indent=2)
print("\nDeploy folder:"); [print("  ", os.path.relpath(str(p), DEPLOY)) for
```

```
Saved artifact at '/content/runs/mobilenetv2_saved_model'. The following enc

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 224, 224, 3), dtype=tf.f
Output Type:
  TensorSpec(shape=(None, 4), dtype=tf.float32, name=None)
Captures:
  134406711981456: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134407245372048: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134407245380496: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134407245373392: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134407245371664: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712221264: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712225680: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712225488: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712225872: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712219728: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712219536: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712227216: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712219920: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712217040: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712227408: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712406480: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712406096: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712406288: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712408016: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712405904: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712395536: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712395728: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712400144: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712399760: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712395920: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712409360: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712409552: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712409936: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712409168: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406712409744: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710968784: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710969552: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710969360: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710969744: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710968592: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710968976: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710970320: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710970128: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710969168: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710970512: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710970896: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710971472: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710971280: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710969936: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710971664: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710972048: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710972816: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710972624: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710973008: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134406710968400: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

11) (Optional but strongly recommended) Test on photos from **your own** region

The public dataset was photographed in Tanzania. A model that scores 98 % there can drop a lot on photos from Pakistani farms (different floors, light, birds, feed). Even **40–100 photos** from a local farm/vet, labelled by a vet, gives you a real-world number and a strong point in your viva.

Create folders on Drive: `PoultryFYP/local_test/Coccidiosis`, `Healthy`, `Newcastle`, `Salmonella` (any subset is fine) and put the photos inside.

11) Evaluate on your local photos (skips itself if the folder does not exist)

```
#@title 11) Evaluate on your local photos (skips itself if the folder does n
if os.path.isdir(LOCAL_TEST_DIR):
    try:
        loc = index_images(LOCAL_TEST_DIR)
        Xl = np.stack([np.asarray(load_resized(p)) for p in loc.src]); yl =
        pl, _, _ = tflite_predict(final_path, Xl)
        print(f"{len(loc)} local photos | accuracy = {accuracy_score(yl, pl.
        present = sorted(set(yl))
        print(classification_report(yl, pl.argmax(1), labels=present, target
        print("Answered 'uncertain':", f"{np.mean([assess(p)['status']=='unc
        plot_eval(pl, yl, "LOCAL_" + EXPORT_MODEL, "TFLite")
    except Exception as e:
        print("Could not evaluate local photos:", e)
else:
    print("No local_test folder found -> skipped. (See the markdown cell abo
```

12) Zip the results and download them to your computer

```
#@title 12) Zip the results and download them to your computer
zip_path = "/content/poultry_results.zip"
with zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as z:
    for p in Path(OUT_DIR).rglob("*"):
        if p.is_file() and p.suffix != ".keras" and "variants" not in p.parts
            z.write(p, p.relative_to(OUT_DIR))
print(f"Created {zip_path} ({os.path.getsize(zip_path)/1e6:.1f} MB): deploy/
print("Your files are also in:", OUT_DIR)
try:
    from google.colab import files
    files.download(zip_path)
except Exception as e:
    print("(Automatic download only works inside Colab.)")
```

```
    Created /content/poultry_results.zip (8.7 MB): deploy/ folder + all figures,
    Your files are also in: /content/PoultryFYP/outputs
```

✅ Done — what to copy into Flutter

Everything for the app is in `PoultryFYP/outputs/deploy/` (also inside the zip you just downloaded):

File	
`poultry_<model>_<variant>.tflite` (normally `poultry_mobilenetv2_fp16.tflite`)	`assets/model/` →
`labels.txt`	class order: index 0..
`severity_config.json`	thresholds for the se
`model_meta.json`	the input/normalisati
`parity/`	8 sample images + tl

The input contract (the most common source of bugs)

- Shape `[1, 224, 224, 3]`, **float32, RGB, values 0–255. No division by 255, no mean/std** — normalisation is inside the model.
- Resize the *whole* photo **directly to 224×224** (no crop, no padding), area/average interpolation; apply EXIF orientation first.
- Output = 4 softmax probabilities in the order of `labels.txt`.

```
import 'dart:typed_data';
import 'package:image/image.dart' as img;
import 'package:tflite_flutter/tflite_flutter.dart';

Float32List preprocess(Uint8List bytes) {
  var im = img.decodeImage(bytes)!;
  im = img.bakeOrientation(im);
  im = img.copyResize(im, width: 224, height: 224, interpolation: img.Interpolat
  final input = Float32List(224 * 224 * 3);
  var i = 0;
  for (var y = 0; y < 224; y++) {
    for (var x = 0; x < 224; x++) {
      final p = im.getPixel(x, y);
      input[i++] = p.r.toDouble();   // RGB, 0..255
      input[i++] = p.g.toDouble();
      input[i++] = p.b.toDouble();
    }
  }
  return input;
}
```

```
Future<List<double>> predict(Interpreter interpreter, Uint8List bytes) async {
  final input = preprocess(bytes).reshape([1, 224, 224, 3]);
  final output = List.filled(4, 0.0).reshape([1, 4]);
  interpreter.run(input, output);
  return List<double>.from(output[0]);
}
// interpreter = await Interpreter.fromAsset('assets/model/poultry_mobilenetv2_f
```

(Package APIs differ slightly between versions — adapt to yours. Decode big photos off the UI thread, e.g. `compute()`*, or ask* `image_picker` *for* `maxWidth: 1024`*.)*

Parity test — do this before trusting the app: run the 8 images in `deploy/parity/` through your app's `predict()`. The probabilities must match `parity/expected.json` to about 0.01. If not: wrong channel order, 0–1 scaling, or a rotated image.

Severity + "Uncertain" rule in Dart (reads `severity_config.json`)

```
const _sev = ['Mild', 'Moderate', 'Severe'];

String? severity(String label, double conf, Map cfg) {
  if (conf < cfg['uncertain_below']) return 'Uncertain';  // ask to retake photo
  if (label == 'Healthy') return null;                    // no severity for hea
  var s = conf < cfg['mild_max'] ? 0 : (conf < cfg['moderate_max'] ? 1 : 2);
  final floor = cfg['min_severity_by_class'][label];      // e.g. Newcastle is r
  if (floor != null && s < _sev.indexOf(floor)) s = _sev.indexOf(floor);
  return _sev[s];
}
```

For your report and viva

Which figure/table goes where (all saved in `outputs/`):